Travel Patterns Between States and Regions

Julian Fuemmeler

Table of Contents

Introduction

The United States is a melting pot of all kinds of people – all with different lifestyles, jobs, habits, etc. Sometimes everyone's lifestyle will converge to a culture whether regionally or on a state level. The goal of this analysis is to take a deeper look into travel patterns amongst people on a state level to see if a significant portion of states can be classified with just a few features.

Discussion of methods

Data Selection

The basis for this analysis is built on data from the U.S. Bureau of Transportation containing a list of Trips per state state. "Trips are defined as movements that include a stay of longer than 10 minutes at an anonymized location away from home. Home locations are inputted on a weekly basis. A movement with multiple stays of longer than 10 minutes before returning home is counted as multiple trips." (2)

For analysis, the data for trips is adjusted for population density of each state to adjust for land area and population for a fair analysis when comparing travel patterns between states. This aims to account for areas with more public transit, large cities in small states, and low populations in larger states.

Additional features are augmented from the trips dataset in the aim of assisting classification model training by giving the models more information on a state level. These include ratios of *short trips vs medium trips, medium trips to long trips, long trips to very long trips* – and the standard deviation of trips between each county in state to give the models data on the spread of travel activity throughout each state.

Features chosen include those which may account for travel patterns – that can make an impact amongst every state that are aspects of individuals or households within those states – not the states themselves. These features are Age, median household income, type of job, type of vehicle (gas, electric, etc.), and amount spent on personal expenditures.

Analysis Strategy

Feature analysis

Features are compared through a correlation matrix to find the relationship amongst features – namely, find the relationship between different features and the number of trips taken. Additionally, features are analyzed by performing hierarchal clustering of the features as classes to further inspect and verify results from the correlation matrix.

Clustering analysis

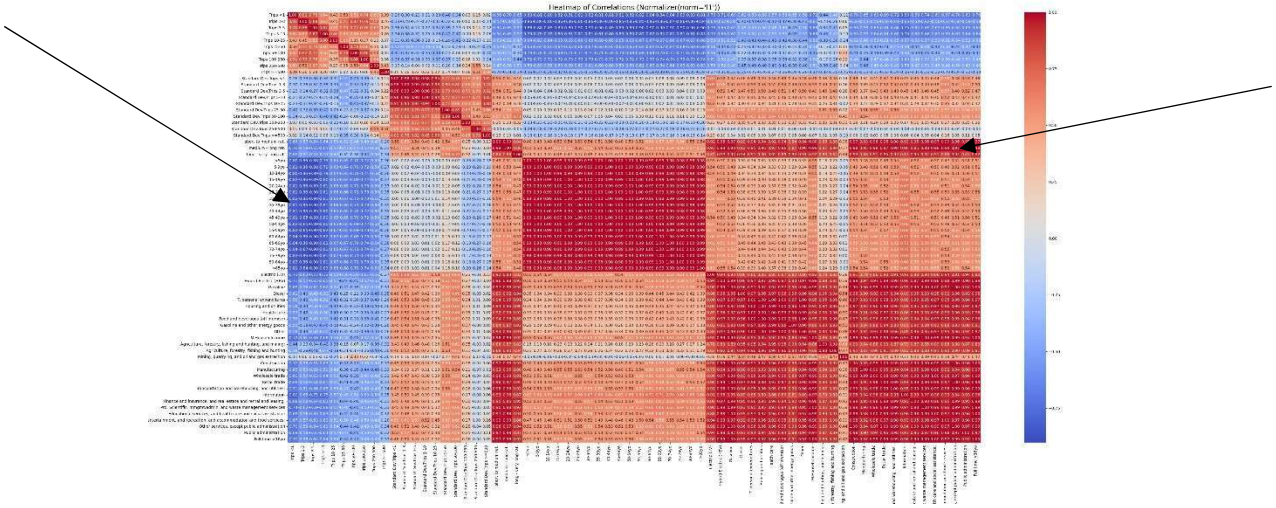
Data is reduced using PCA and clustered using Spectral, Hierarchical, and K-means clustering methods. States are labeled across regional bounds to see if clustering methods group in a way that is sensible. This was done a few times in analysis to inform additional features that may be necessary to accurately classify the set. Clustering is done via two sets a goal number of clusters. These sets being a lower set [5, 9, 20] and a higher set [30, 40, 49] each to (a) describe relationships among states and compare the clusters with the state's respective region and (b) compare clusters with other bounds including economic, population, cultural bounds.

Classification

Classification is done with multiple methodologies including Random Forest, Gradient Boosting, Gaussian Naïve Bayes, SVM RBF. Classifiers are used to test how well the clusters describe a group of states analysis.

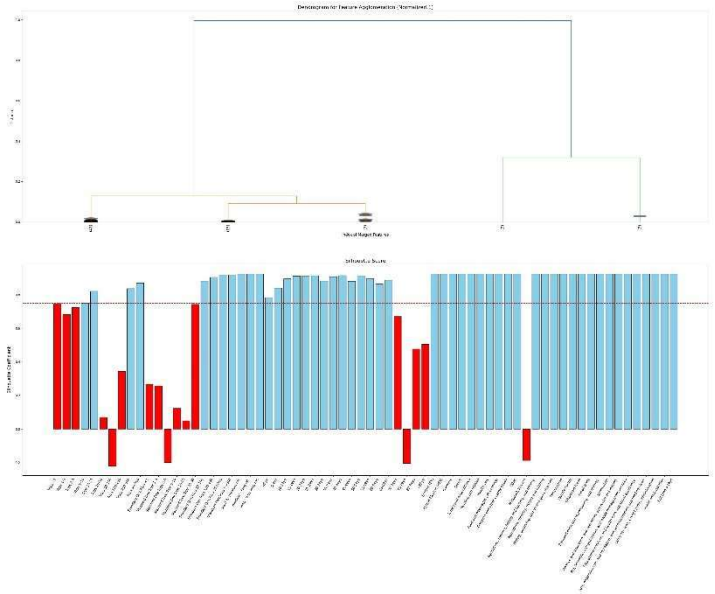
Results, Data Visualization, and Analysis

Feature Analysis



Most notably on the list of features, though hard to see, is a strong positive correlation between the augmented features, the *ratios of short trips vs medium trips, medium trips to long trips*, and strong *negative* correlation between the age groups and the number of trips. Though neither are correlated with the same types of trip labels.

The right graph shows the most correlated features clustered with hierarchical clustering to see if the features would be able to find their respective homes.

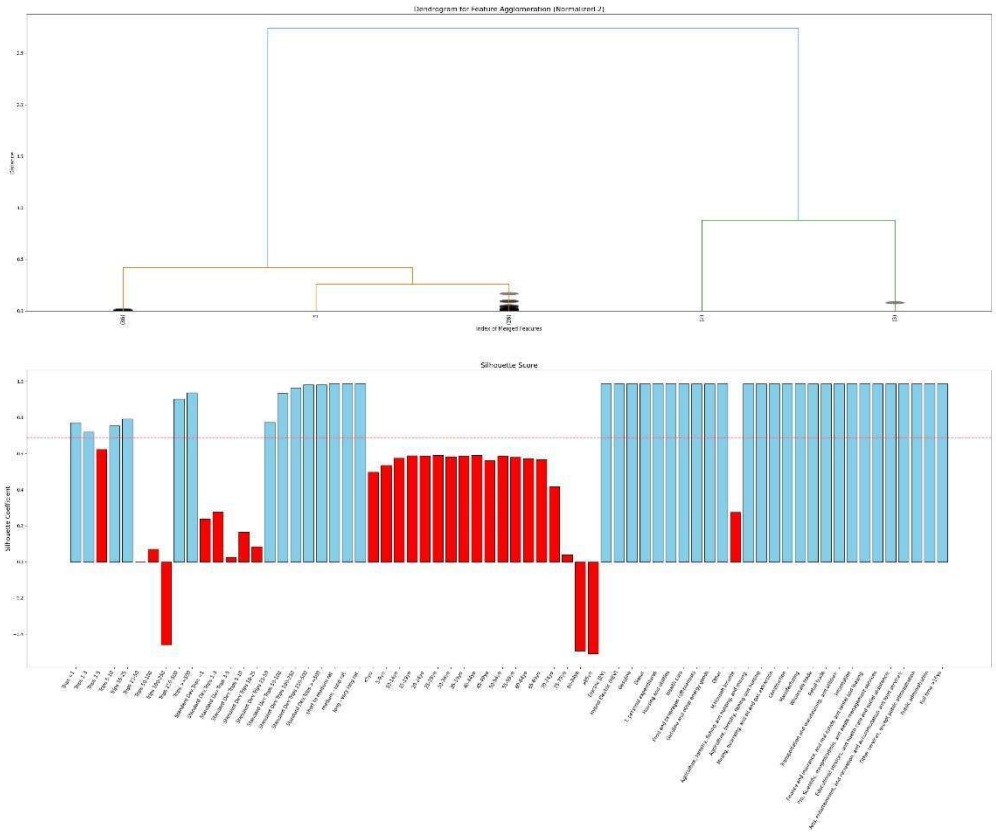


The trips themselves have some depravities with medium length trips 25-100 miles dropping below the average, and even into the negative, nearly correlating with the augmented features of standard deviation. Indicating that either the standard deviations of trips among states vs the number of trips may amount to about the same or short to medium length trips are about the same number across states.

Additionally, the older ages having some issues fitting into their group – this could potentially

be because the number of people in their older ages across states average out to be able the same, and Median Household Income was unable to find its cluster.

For the most part – all other age groups and all jobs were able to find their cluster quite well with a silhouette score decently above the average. Indicating these features will most likely be decent for classification, though the disparities among trips may make classification difficult.



Normalizing by L2 however, shows the age inching well below the median – with older ages falling off ever more than they did before. Types of jobs, however, have an even higher silhouette score at exactly 1.0. Shorter trips were able to find their groups well, with only the medium – long trips falling off, and shorter trips with scores around ~0.2.

Conclusion – Feature Analysis

Given the data, I train models on two sets of data – one including age, and one without age. Both sets of data have shorter length standard deviations dropped as neither performed well in the initial clustering of the features.

Clustering Analysis – Overview

The best performing dataset for clustering was the dataset where Age was excluded from the features. Fig. 1 shows the spread among states after PCA, and cluster labels were assigned. This type of grouping among classes occurred with all clustering methods at clustering sizes– indicating that the spread of age groups among states largely equalizes to be similar except for a few outliers, making age not a good feature for distinguishing among states.

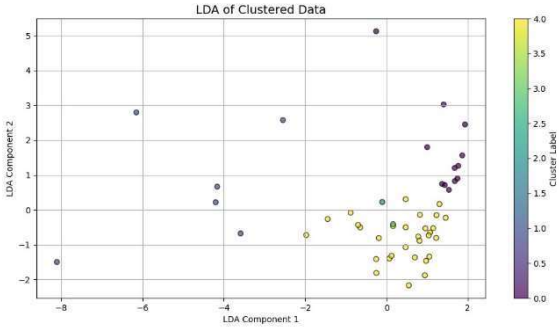
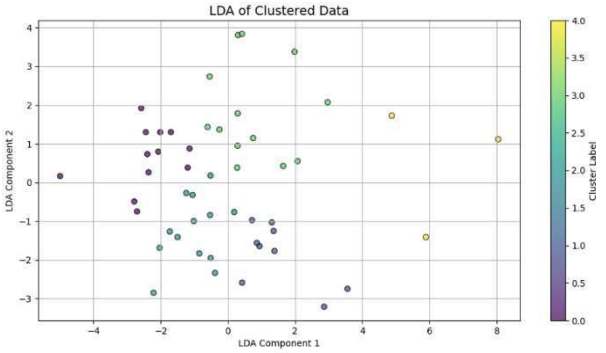
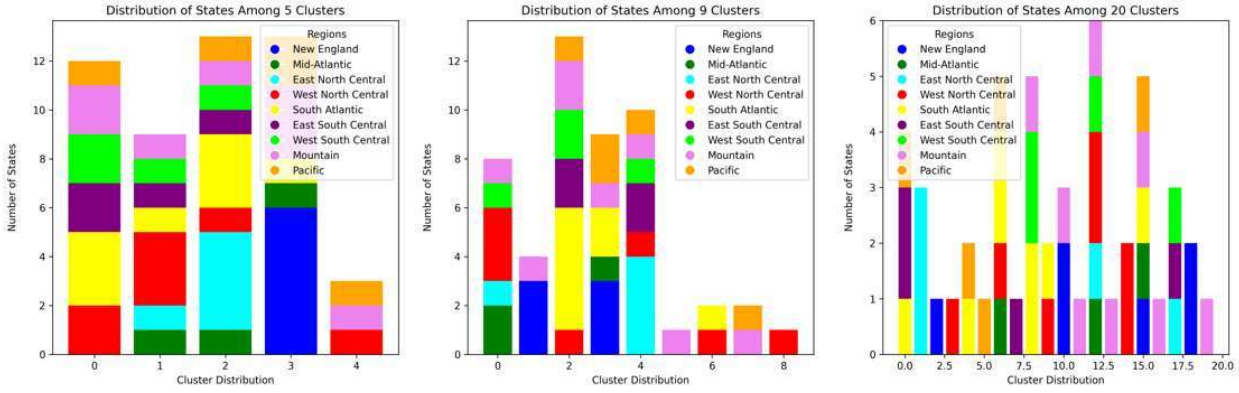
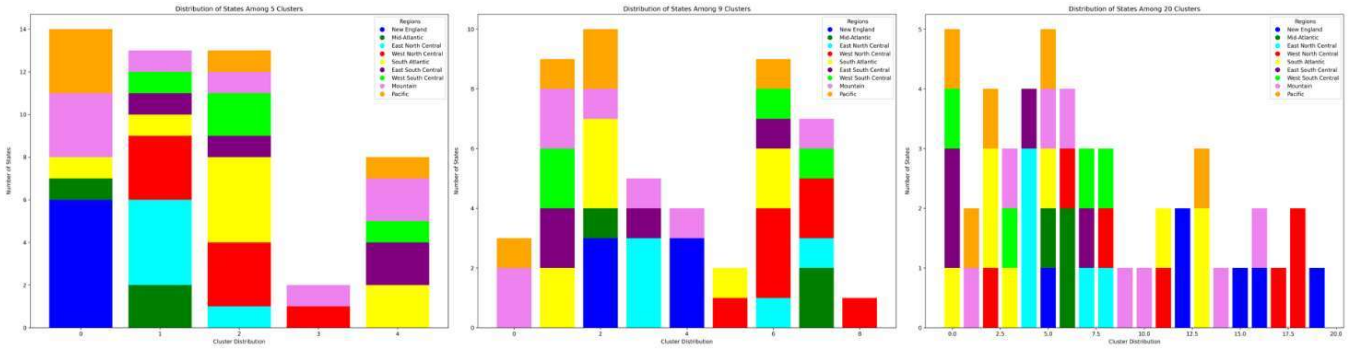


Fig 2. Shows the spread among states without age being considered as a feature, and the spread among states increases substantially, the spread increases and decreases varyingly across cluster methods, but every set that did not include age as a feature significantly outperformed sets with age included. So, for the rest of analysis – only sets with age included will be discussed outside of a few comparisons.

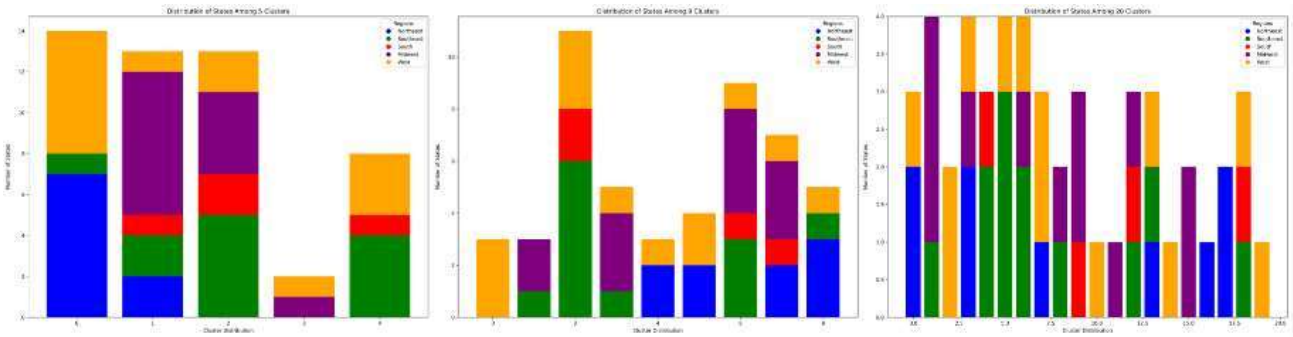


Clustering Analysis – 5,9,20 clusters





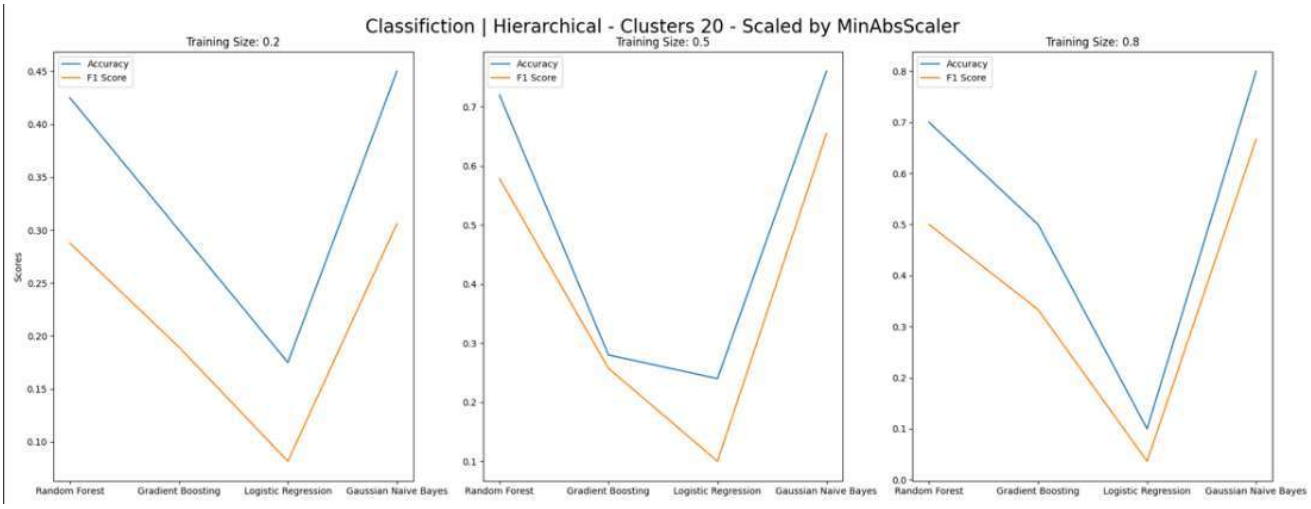
K-Means and Hierarchical Clustering performed similarly well, each being able to distinguish among regions at a relatively decent accuracy – especially when taking a closer look at tighter regions defined by the U.S. Census Bureau – South, Northeast, Midwest, and West. Even up to 20 clusters, K-Means a Hierarchical Clusters were able to pick out relative regions between states but had trouble distinguishing western/pacific states in both forms of representation. Indicating high variability of patterns between those states. New England states, however, were the most clustered states, with both hierarchical and K-Means grouping all 6 New-England states within the same cluster on 5 clusters.

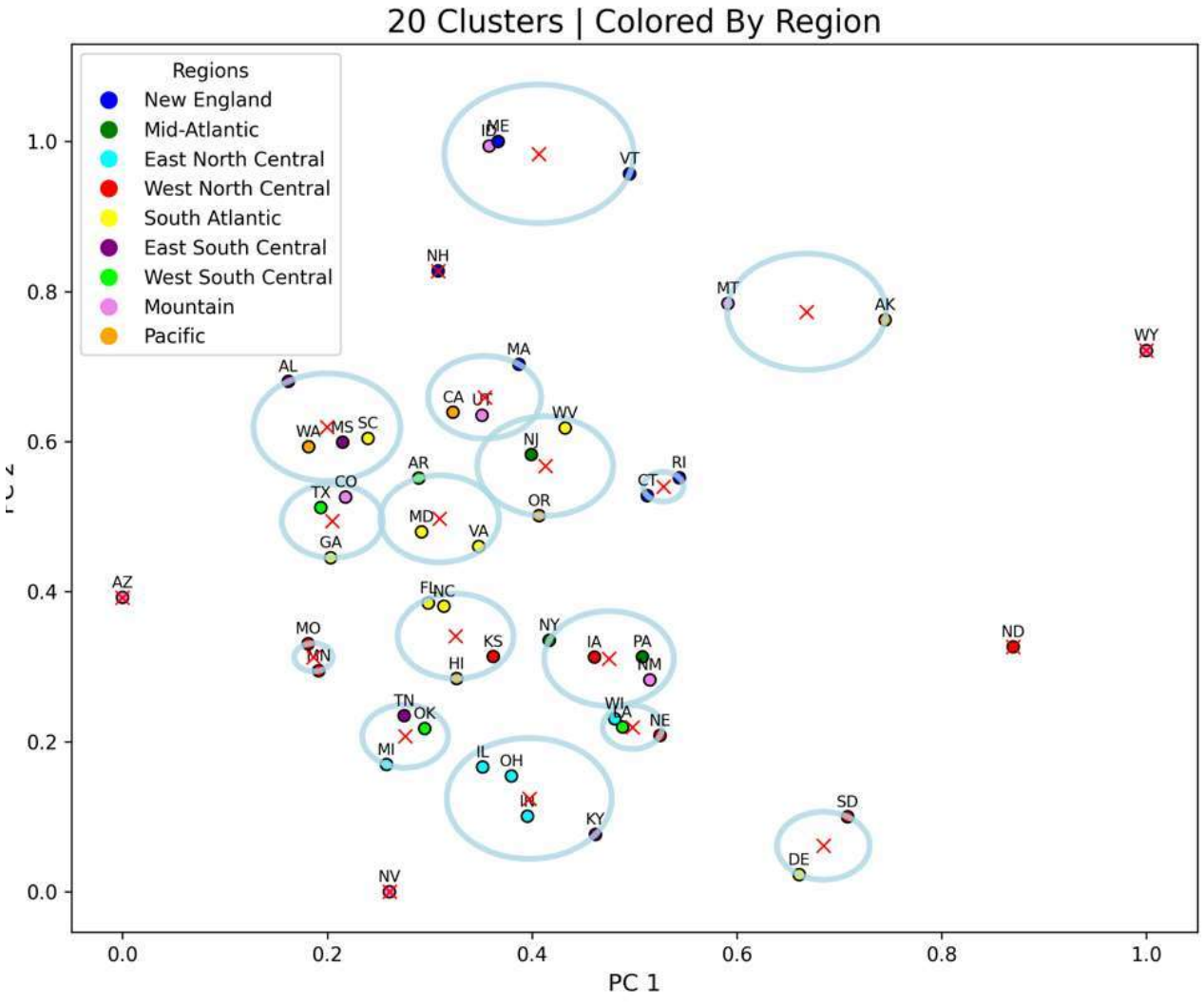


Spectral clustering however – had a maximum discrepancy among clusters at 20 and a minimum discrepancy among clusters at 10. Leading to extremely low f1 scores in classification model training.

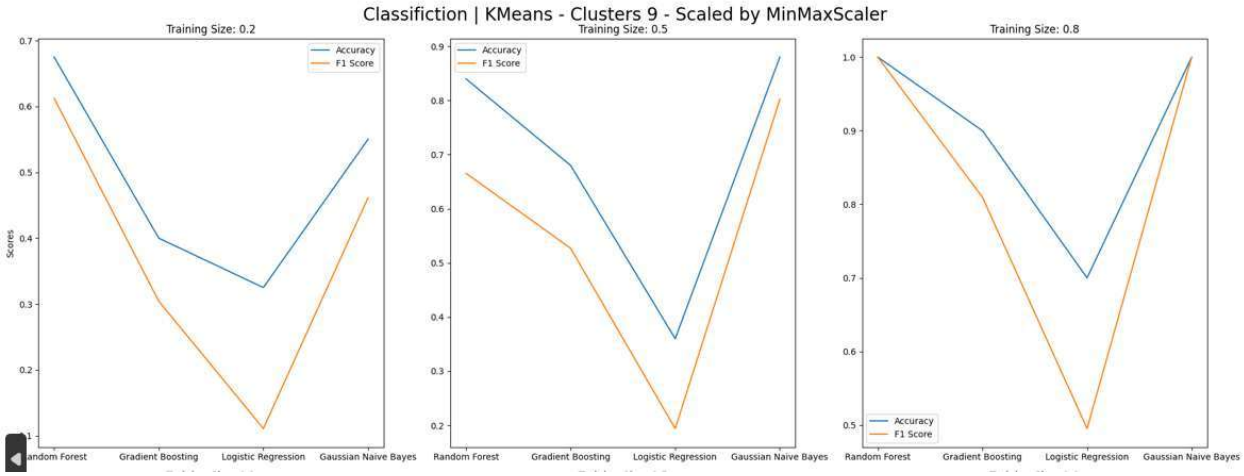
Classification

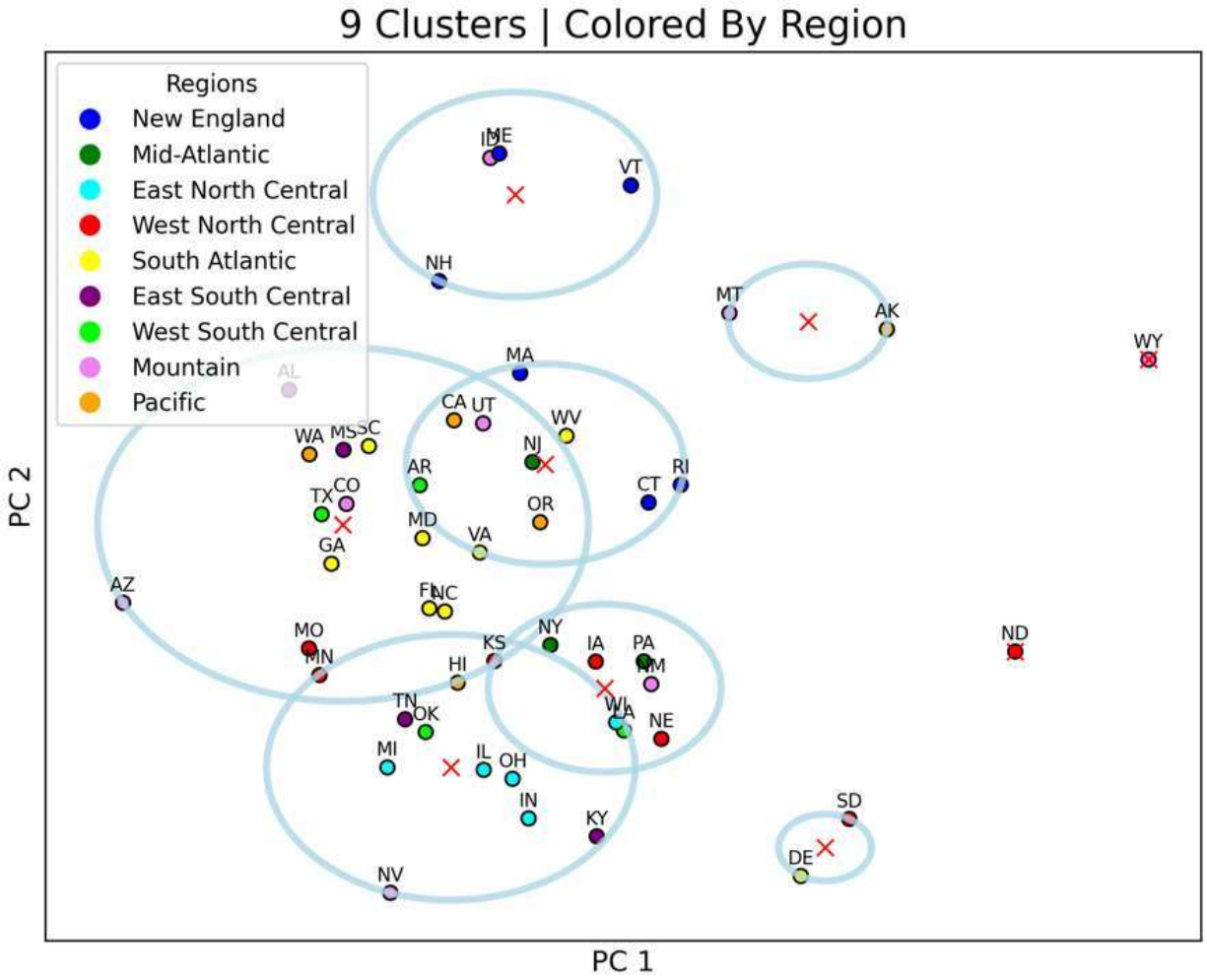
Hierarchical clustering was able to outperform Keans overall in with training classifications models, being the only model at 20 clusters able to produce a decent accuracy to f1 score ratio with the Gaussian Naïve Bayes classifier being able to distinguish the best among clusters.





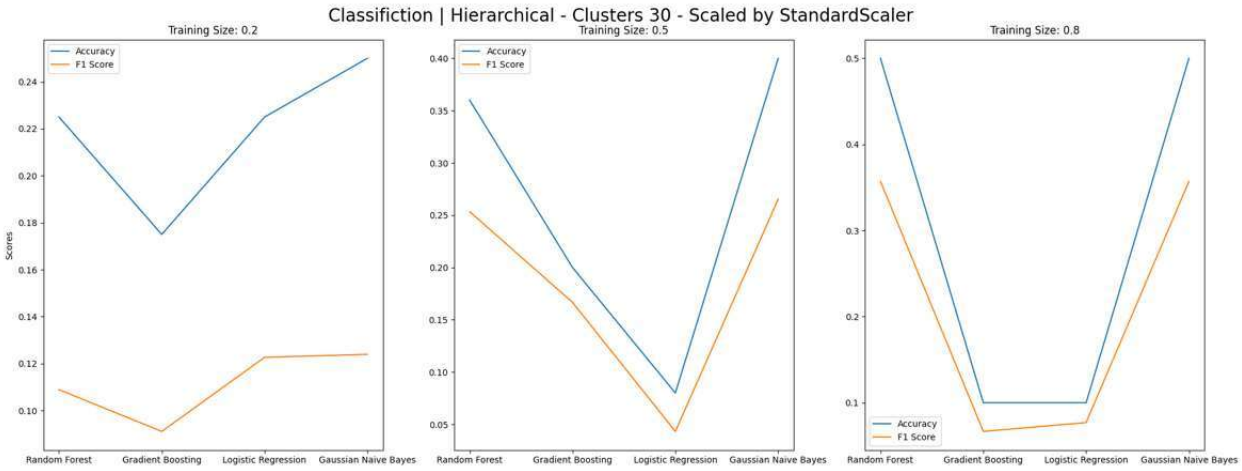
However, the only model that was able to achieve 100 % accuracy – along with 100 % f1 score was K-Means on both 5 clusters and 9 clusters, with both Random Forest and Gaussian Naïve Bayes Classifiers.

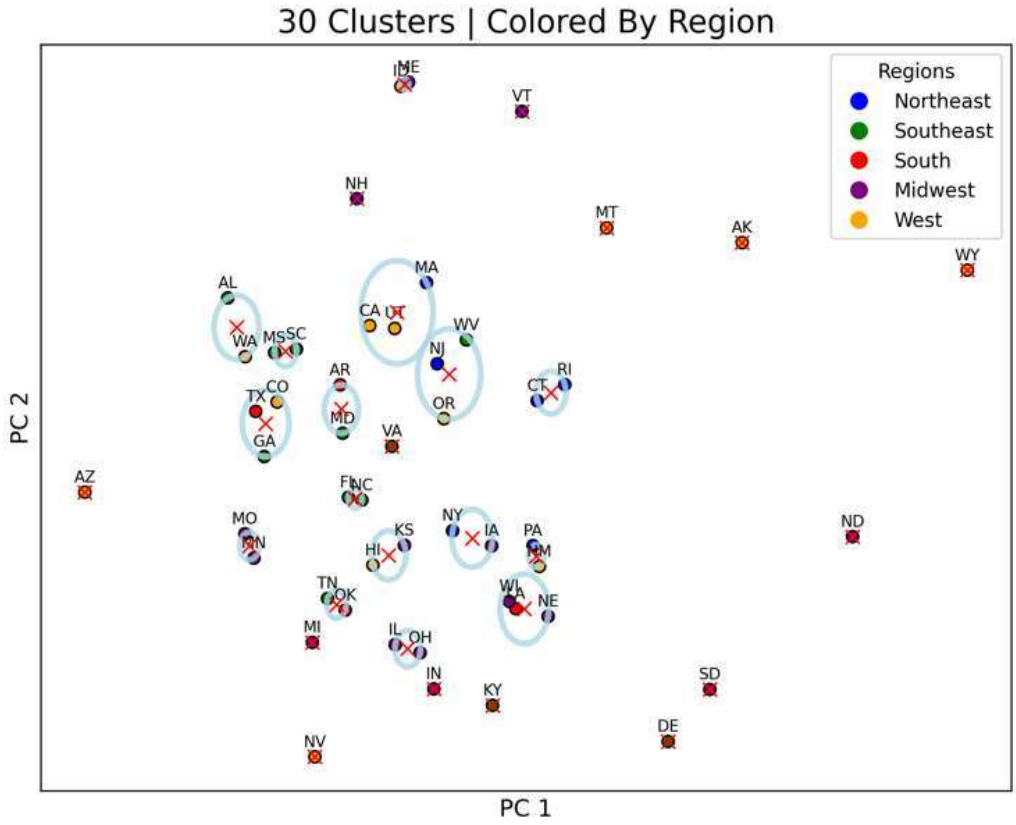




Clustering Analysis – 30, 40, 50 clusters

No model was able to able accurately distinguish between states at any point from 30 to 50 clusters. The best performing was again hierarchical clustering using Random Forest and Gaussian Naïve Bayes classifiers with a 50% accuracy and ~35% f1 score.





Conclusion

Further analysis and data would need to be gathered to find simple features that have enough variability between states to be able to distinguish among all 50 states. However, there is a decent degree of variability in working lifestyles between regions of states, enough for a model to be able to group states regionally with some accuracy. The Gaussian Naïve Bayes classifier consistently gave some of the higher f1 and accuracy scores indicating that the clusters found for each state were distinguishable.

An additional analysis could be done the state with outliers removed – Texas, Florida, California, New York, Alaska, Hawaii – and between the outliers themselves to potentially get a better insight into what differentiating factors between states that have unique populations vs states whose populations make up the average in the United States.

References

1. Bureau of Transportation Statistics. (n.d.). Trips by Distance. Retrieved 4/28/2024 from https://data.bts.gov/Research-and-Statistics/Trips-by-Distance/w96p-f2qv/data_preview

2. Publisher Bureau of Transportation Statistics. (2023, February 1). *Department of Transportation - trips by distance*. Catalog. https://catalog.data.gov/dataset/trips-by-distance

3. U.S. Census Bureau. "Industry by Sex for the Full-Time, Year-Round Civilian Employed Population 16 Years and Over." *American Community Survey, ACS 1-Year Estimates Subject Tables, Table S2404*, 2022, https://data.census.gov/table/ACSST1Y2022.S2404?g=010XX00US$0400000. Accessed on April 28, 2024.

4. U.S. Census Bureau. "TOTAL POPULATION." *Decennial Census, DEC 118th Congressional District Summary File, Table P1*, 2020, https://data.census.gov/table/DECENNIALCD1182020.P1?g=010XX00US$0400000. Accessed on April 29, 2024.

5. U.S. Census Bureau. "Age and Sex." *American Community Survey, ACS 1-Year Estimates Subject Tables, Table S0101*, 2022, https://data.census.gov/table/ACSST1Y2022.S0101?q=population by age by state. Accessed on April 29, 2024.

Appendix

```
'''
TravelDistance:    Class to aggregate , test, run analytics on different feature
sets in /Subsets.

    function defintions:
        load_travel_data:    loads data from 'trips_data' given the parameters
different features  / attach features to class labels (i.e. regions and counties)
            __growth_rate:    calculates growth rate of number of trips between
years relative to the original year in trips_data
        load_feature_data:   loads features features from /Subset - returns a
dataframe with original class labels and new features attached
'''
import os
cwd = os.getcwd()
os.chdir(os.path.dirname(os.path.abspath(__file__)))
import numpy as np
import pandas as pd
from Travel.aggregate import Aggregation

per_capita_adjustments = ['vehicles','personal_expenditures', 'employment_types']

class TravelDistance:
    def __init__(self):
        self.trips_data = None
        self.areas = ["State", "Region"]
        self.regions = Aggregation.regions_abrv
        self.trips_distance = ['Trips <1', 'Trips 1-3', 'Trips 3-5', 'Trips 5-
10', 'Trips 10-25', 'Trips 25-50', 'Trips 50-100', 'Trips 100-250', 'Trips 250-
500', 'Trips >=500']
        self.feature_data = ["airports", "gdp", "household_income", "poverty",
"vehicles", "personal_expenditures", "age", "population_totals",
'employment_types']

    def load_travel_data(self, area = "State", year = "", metric = "", addstate =
False, per_capita_adjustment = False, denisty_adjustment=False, **kwargs) ->
pd.DataFrame:
        data = pd.read_csv("../trips_data.csv")
        data = data[(data[self.trips_distance] != 0).any(axis=1)]
        data['Date'] = data['Date'].astype('str')

        if year != 'all':
            if isinstance(year, list):
                data = data[data['Date'].isin(year)]
```

```
            elif isinstance(year, range):
                data = data[data['Date'].between(str(min(year)), str(max(year)))]
            else:
                data = data[data['Date'] == str(year)]

        if metric == "growth":
            g_data = self.__growth_rate(data, area, year, addstate, **kwargs)
            self.trips_data = g_data
            return g_data

        if area == 'Counties':
            data_grouped = data.groupby(['State','Counties'], as_index=False)
            for trip_category in ['Trips <1', 'Trips 1-3', 'Trips 3-5', 'Trips 5-
10', 'Trips 10-25', 'Trips 25-50', 'Trips 50-100', 'Trips 100-250', 'Trips 250-
500', 'Trips >=500']:
                data['Standard Dev.' + trip_category] =
data.groupby('State')[trip_category].transform('std')

            std_dev = data[['Standard Dev.Trips <1', 'Standard Dev.Trips 1-3',
                            'Standard Dev.Trips 3-5', 'Standard Dev.Trips 5-10',
                            'Standard Dev.Trips 10-25', 'Standard Dev.Trips 25-
50',
                            'Standard Dev.Trips 50-100', 'Standard Dev.Trips 100-
250',
                            'Standard Dev.Trips 250-500', 'Standard Dev.Trips
>=500']]


            data.drop(std_dev.columns,inplace=True,axis=1)
            std_dev.index = data['State']
            std_dev.drop_duplicates(inplace=True)
            print(std_dev)
            data_grouped = data.groupby(['State'])

        elif area == 'State':
            data_grouped = data.groupby(['State'])
        elif area == 'Region':
            data['Region'] = data['State'].map({state: region for region, states
in self.regions.items() for state in states})
            if addstate:
                data_grouped = data.groupby(['State','Region'])
            else:
                data_grouped = data.groupby('Region')
        else:
            raise ValueError(f"No area {area} exist for this dataset")
```

```
if metric == "total":
    data = data_grouped[self.trips_distance].sum()
    print(data)
elif metric == "average":
    data = data_grouped[self.trips_distance].mean()
else:
    raise ValueError(f'No metric {metric} exist for this dataset')

data.sort_index(inplace=True)
data.reset_index(inplace=True)
data.index = data['State']

if per_capita_adjustment:
    if area == 'Counties':
        data = data.join(std_dev, how="outer")

    pop = pd.read_csv("../population_totals.csv")
    pop.index = pop['State']
    pop = Aggregation.adjust_source_datatypes(pop)
    pop.index = pop["State"]
    pop.drop("State",axis=1, inplace=True)
    data.drop("State",axis=1,inplace=True)
    data, pop = data.align(pop, join='inner', axis=0)
    data = data.div(pop['Pop. 2022'], axis=0)

elif denisty_adjustment:
    if area == 'Counties':
        data = data.join(std_dev, how="outer")
    dens = pd.read_csv("../population_density.csv")
    dens.index = dens['State']
    dens = Aggregation.adjust_source_datatypes(dens)
    dens.index = dens["State"]
    dens.drop("State",axis=1, inplace=True)
    data.drop("State",axis=1,inplace=True)
    data, pop = data.align(dens, join='inner', axis=0)
    print(data)
    print(data.columns)
    data = data.div(dens['Density'], axis=0)
```

```
        data['short to medium rat.'] = data['Trips 3-5'] / data['Trips 25-50']
        data['medium - long rat.'] = data['Trips 25-50'] / data['Trips 50-100']
        data['long - very long rat.'] = data['Trips 50-100'] / data['Trips 250-
500']


        data.replace([np.inf, -np.inf], np.nan, inplace=True)
        data.fillna(0, inplace=True)  # Or some other strategy to handle NaNs


        self.trips_data = data
        return data

    def __growth_rate(self, data, area, year, addstate, **kwargs):
        if area == "State":
            group_string = "State"
        elif area == "Counties":
            group_string = "Counties"
        elif area == "Region":
            data['Region'] = data['State'].map({state: region for region, states
in self.regions.items() for state in states})
            group_string = "Region"
        else:
            raise ValueError(f"No area {area} found in dataset")

        if addstate and area != "State":
            group = [group_string,"State", "Date"]
            grouped= data.groupby(group)[self.trips_distance].sum()
            group.remove("Date")
            grouped.reset_index(inplace=True)
            initial = grouped.groupby(group).first().reset_index()
            final = grouped.groupby(group).last().reset_index()
        else:
            grouped=data.groupby([group_string,'Date'])[self.trips_distance].sum(
)
            grouped.reset_index(inplace=True)
            initial = grouped.groupby(group_string).first().reset_index()
            final = grouped.groupby(group_string).last().reset_index()

        results = pd.DataFrame()
        for trip in self.trips_distance:
```

```
            results[f'Growth_{trip}'] = ((final[trip] - initial[trip]) /
initial[trip]) * 100

        if addstate is False:
            results.set_index(initial[area], inplace=True)
        else:
            results.set_index(initial[group[0]], inplace=True)
            results.reset_index(inplace=True)
            results.set_index(initial[group[1]], inplace=True)

        return results

    def load_feature_data(self, include = [], population_adjustment = False) ->
pd.DataFrame:


        for dataset in include:
            feature_df = pd.read_csv(f'../Subsets/{dataset}.csv')
            df = Aggregation.adjust_source_datatypes(feature_df)
            df = Aggregation._drop_excess_labels(df)

            df.index = df['State']
            df.drop('State',axis=1,inplace=True)

            if dataset in per_capita_adjustments and population_adjustment:
                    pop = pd.read_csv("../population_totals.csv")
                    pop.index = pop['State']
                    pop = Aggregation.adjust_source_datatypes(pop)
                    pop.index = pop["State"]
                    pop.drop("State",axis=1, inplace=True)
                    df, pop = df.align(pop, join='inner', axis=0)
                    df = df.div(pop['Pop. 2022'], axis=0)


            if 'State' in self.trips_data.columns:
                self.trips_data.index = self.trips_data['State']
                self.trips_data.drop('State', axis=1,inplace=True)

            self.trips_data = self.trips_data.join(df, how="outer")

        self.trips_data =
Aggregation._drop_excess_labels(self.trips_data).dropna(axis=0)
        return self.trips_data

'''
```

```
Aggregation.py    :  Aggregation dictionaries, functions to free space in
TravelDistance.py
special note: some stuff in here is used in TravelDistance.py

'''

import os
os.chdir(os.path.dirname(os.path.abspath(__file__)))
import pandas as pd


northeast_abbreviations = ['CT', 'ME', 'MA', 'NH', 'RI', 'VT', 'NJ', 'NY', 'PA']
southeast_abbreviations = ['AL', 'AR', 'DE', 'FL', 'GA', 'KY', 'LA', 'MD', 'MS',
'NC', 'OK', 'SC', 'TN', 'TX', 'VA', 'WV']
south_abbreviations = ['AZ', 'NM', 'TX', 'OK', 'AR', 'LA', 'MS', 'AL', 'FL',
'GA', 'SC', 'NC', 'TN']
midwest_abbreviations = ['IL', 'IN', 'IA', 'KS', 'MI', 'MN', 'MO', 'NE', 'ND',
'OH', 'SD', 'WI']
west_abbreviations = ['AK', 'CA', 'CO', 'HI', 'ID', 'MT', 'NV', 'OR', 'UT', 'WA',
'WY']
regions_abrv = {
    'Northeast': northeast_abbreviations,
    'Southeast': southeast_abbreviations,
    'South': south_abbreviations,
    'Midwest': midwest_abbreviations,
    'West': west_abbreviations
}

DROP_INDEX = ['United States', 'District of Columbia', 'Puerto Rico']

STATE_DICT = {'AL': 'Alabama','AK': 'Alaska','AZ': 'Arizona','AR': 'Arkansas',
'CA': 'California', 'CO': 'Colorado', 'CT': 'Connecticut', 'DE': 'Delaware',
'DC': 'District of Columbia', 'FL': 'Florida',
    'GA': 'Georgia', 'HI': 'Hawaii', 'ID': 'Idaho','IL': 'Illinois', 'IN':
'Indiana', 'IA': 'Iowa', 'KS': 'Kansas', 'KY': 'Kentucky','LA': 'Louisiana','ME':
'Maine',
    'MD': 'Maryland','MA': 'Massachusetts','MI': 'Michigan','MN': 'Minnesota',
'MS': 'Mississippi','MO': 'Missouri','MT': 'Montana','NE': 'Nebraska','NV':
'Nevada',
    'NH': 'New Hampshire','NJ': 'New Jersey','NM': 'New Mexico','NY': 'New
York','NC': 'North Carolina', 'ND': 'North Dakota', 'OH': 'Ohio', 'OK':
'Oklahoma', 'OR': 'Oregon',
    'PA': 'Pennsylvania', 'RI': 'Rhode Island', 'SC': 'South Carolina', 'SD':
'South Dakota', 'TN': 'Tennessee', 'TX': 'Texas', 'UT': 'Utah', 'VT': 'Vermont',
'VA': 'Virginia',
```

```
    'WA': 'Washington',  'WV': 'West Virginia',  'WI': 'Wisconsin', 'WY':
'Wyoming'
}

REVERSE_STATE_DICT = {}
for key, value in STATE_DICT.items():
    REVERSE_STATE_DICT[value] = key

def adjust_source_datatypes(dataset) -> pd.DataFrame:
    dataset['State'] = dataset['State'].map(REVERSE_STATE_DICT)
    dataset.index = dataset['State']
    dataset.replace(',', '', regex=True, inplace=True)

    new = dataset.apply(pd.to_numeric, errors="coerce")
    new.dropna(how="all", axis=1, inplace=True)
    new['State'] = dataset['State']
    new.dropna(how="any", axis=0, inplace=True)
    return new

def _drop_excess_labels(df):
    drop_idx = DROP_INDEX
    df.drop(index=drop_idx, inplace=True, errors='ignore')
    df.sort_index(inplace=True)

    return df

'''
main.py:
    Functions pertaining to feature analysis annd calling clustering methods from
Clustering.py
    plot_feature_analysis - create heatmap and cluster features as classes to
find how seperable features are from each other
    cluster_data - call functions from clustering.py givin a list of scalers and
datasets
    feature_data - do feature analysis given a list of scalers and datasets
'''
import os
os.chdir(os.path.dirname(os.path.abspath(__file__)))
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.metrics import silhouette_samples
from sklearn.preprocessing import (StandardScaler, MinMaxScaler,
```

```
                                    RobustScaler, MaxAbsScaler)

from sklearn.decomposition import PCA
from sklearn.cluster import ( AgglomerativeClustering, FeatureAgglomeration)
from Travel.TravelDistance import TravelDistance
from Clustering import k_means_plot_data, sepctra_plot_data,
hierarchical_plot_data
import seaborn as sns




def plot_feature_analysis(X, scaler, scale_name, directory="", poly = False):
    if scaler:
        scaled_data = scaler.fit_transform(X)
    feature_names=X.columns
    plt.figure(figsize=(40, 20))
    sns.heatmap(pd.DataFrame(scaled_data, columns=feature_names).corr(),
annot=True, fmt=".2f", cmap='coolwarm')
    plt.title(f'Heatmap of Correlations ({scaler})',fontsize=18)
    plt.savefig(f"{directory}/heatmap_{scaler}.png")
    plt.subplots_adjust(left=0.2, right=0.8, top=0.8, bottom=0.2)
    plt.close()

    pca = PCA(n_components=min(len(feature_names), 10))
    pca_data = pca.fit_transform(scaled_data)
    plt.figure(figsize=(12, 8))
    plt.bar(range(1, len(pca.explained_variance_ratio_) + 1),
pca.explained_variance_ratio_)
    plt.ylabel('Explained Variance',fontsize=20, labelpad=16)
    plt.xlabel('Principal Components',fontsize=20, labelpad=16)
    plt.xticks(fontsize=10)
    plt.yticks(fontsize=10)
    plt.title(f'PCA Explained Variance Ratio ({scale_name})',fontsize=18)
    plt.savefig(f"{directory}/pca_explained_variance_{scale_name}.png")
    plt.close()

    if pca_data.shape[1] >= 2:
        plt.figure(figsize=(12, 8))
        plt.scatter(pca_data[:, 0], pca_data[:, 1])
        plt.xlabel('PC1', fontsize=20, labelpad=16)
        plt.ylabel('PC2', fontsize=20, labelpad=16)
        plt.xticks(fontsize=10)
        plt.yticks(fontsize=10)
        plt.title(f'PCA Scatter Plot ({scale_name})',fontsize=18)
```

```
        plt.savefig(f"{directory}/pca_scatter_{scale_name}.png")
        plt.close()

    n_clusters = 5
    agglo = FeatureAgglomeration(n_clusters=n_clusters)
    agglo_data = agglo.fit_transform(scaled_data.T)
    feature_cluster = AgglomerativeClustering(n_clusters=n_clusters,
linkage='ward')
    feature_labels = feature_cluster.fit_predict(agglo_data)

    linkage_matrix = linkage(agglo_data, method="ward")
    silhouette_vals = silhouette_samples(agglo_data, feature_labels)

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(40, 30))

    ax1.set_title(f'Dendrogram for Feature Agglomeration ({scale_name})',
fontsize=16)
    dendrogram( linkage_matrix, truncate_mode='lastp', p=n_clusters,
leaf_rotation=90., leaf_font_size=12., show_contracted=True, ax=ax1
    )
    avg_silhouette = np.mean(silhouette_vals)
    outlier_threshold = avg_silhouette
    colors = ['red' if s < outlier_threshold else 'skyblue' for s in
silhouette_vals]
    ax1.set_xlabel('Index of Merged Features',fontsize=12)
    ax1.set_ylabel('Distance',fontsize=12)
    ax2.set_title(f'Silhouette Score',fontsize=16)
    ax2.bar(range(len(silhouette_vals)), silhouette_vals, color=colors,
edgecolor="black")
    ax2.axhline(y=avg_silhouette, color='r', linestyle='--')  # Average
silhouette score line
    ax2.set_xticks(range(len(feature_names)))
    ax2.set_xticklabels(feature_names,rotation=60, ha='right', fontsize=11)
    ax2.set_ylabel('Silhouette Coefficient',fontsize=14)
    plt.subplots_adjust(bottom=0.18)  # Adjust the value as needed
    plt.savefig(f"{directory}/combined_dendrogram_ward_silhouette_{scale_name}.pn
g")
    plt.close(fig)




def cluster_data(datas, name_scalers, directory_add="", drop = None):
    for name,data in datas:
```

```
        if drop is not None:
            print(data)
            print(data.columns.tolist())
            data = data[drop]
        n_clusters = [[5,9,20],[30,40,50]]
        for i,cluster in enumerate(n_clusters):
            directory_name = f"{directory_add}{cluster}{name}"
            if not os.path.exists(directory_name):
                os.makedirs(directory_name)
            k_means_plot_data(data, components=2, reduce = True,
n_clusters_options = cluster, scalers = name_scalers, directory_name =
directory_name)
            hierarchical_plot_data(data, components=2, reduce=True,
linkage_type="ward", n_clusters_options = cluster, scalers=name_scalers,
directory_name = directory_name, do_features = False)
            sepctra_plot_data(data,components=2,
reduce=True,n_clusters_options=cluster, scalers=name_scalers, directory_name =
directory_name)

def feature_data(datas, name_scalers, directory_add="", poly = False):
    for name,data in datas:
        for s_name, scaler in name_scalers:
            directory_name = f"{directory_add}"
            if not os.path.exists(directory_name):
                os.makedirs(directory_name)
            plot_feature_analysis(data, scaler, scale_name=s_name,
directory=directory_name)




trips_data = ['Trips <1', 'Trips 1-3', 'Trips 3-5', 'Trips 5-10', 'Trips 10-25',
'Trips 25-50', 'Trips 50-100', 'Trips 100-250',
              'Trips 250-500', 'Trips >=500', 'Housing and utilities', 'Gasoline
and other energy goods', 'Gasoline',
              'long - very long rat.', 'medium - long rat.', 'short to medium
rat.']
name_scalers = [("StandardScaler", StandardScaler() ), ("RobustScaler",
RobustScaler(unit_variance=True)),("MinMaxScaler", MinMaxScaler()),
("MinAbsScaler",MaxAbsScaler())]



td = TravelDistance()
cars = ["Electric (EV)", "Hybrid Electric (HEV)", "Gasoline","Diesel"]
```

```
trips = ['Trips <1', 'Trips 1-3', 'Trips 3-5', 'Trips 5-10', 'Trips 10-25',
'Trips 25-50', 'Trips 50-100', 'Trips 100-250', 'Trips 250-500', 'Trips >=500']
ages = [
    "5-9yo", "10-14yo", "15-19yo", "20-24yo", "25-29yo",
    "30-34yo", "35-39yo", "40-44yo", "45-49yo", "50-54yo",
    "55-59yo", "60-64yo", "65-69yo", "70-74yo", "75-79yo",
    "80-84yo", ">85yo"
]
include_all=["age", "vehicles", "personal_expenditures", 'employment_types']
median_ic = ["M.Househ.Income"]
expend = ["T. personal expenditures","Housing and utilities","Health care","Food
and beverages (off-premise)","Gasoline and other energy goods","Other"]
work = [
    'Agriculture, forestry, fishing and hunting', 'Mining, quarrying, and oil and
gas extraction', 'Construction', 'Manufacturing', 'Wholesale trade', 'Retail
trade', 'Transportation and warehousing, and utilities:', 'Information', 'Finance
and insurance, and real estate and rental and leasing:', 'Pro, Scientific,
mmgmt/admin, and waste management services', 'Educational services, and health
care and social assistance:', 'Arts, entertainment, and recreation, and
accommodation and food services:', 'Other services, except public
administration', 'Public administration', 'Full time >16yo'
]
age_work_ratios = ['long - very long rat.', 'medium - long rat.', 'short to
medium rat.'] + ages + work
st = "Standard Dev."



densTpopT = td.load_travel_data(area="Counties", year="2022", metric = "total",
addstate=True, denisty_adjustment = True)
densTpopT = td.load_feature_data(include=include_all, population_adjustment =
True)
datas = [("TTdens__", densTpopT)]
#cluster_data(datas, name_scalers, directory_add="ClusteringAnalysis_AGE_L2")

densTpopT = td.load_travel_data(area="Counties", year="2022", metric = "total",
addstate=True, denisty_adjustment = True)
densTpopT = td.load_feature_data(include=include_all, population_adjustment =
True)

densTpopT.drop(trips, inplace=True, axis=1)
datas = [("TTdens__", densTpopT)]
```

```
name_scalers = [("StandardScaler", StandardScaler() ), ("RobustScaler",
RobustScaler(unit_variance=True)),("MinMaxScaler", MinMaxScaler()),
("MinAbsScaler",MaxAbsScaler())]
cluster_data(datas, name_scalers, directory_add="ClusteringAnalysis_NOAGE_L1")
'''
Clustering.py
    Faciliate plotting, clustering, and calling classification to classify the
clusters

    k_means_plot_data : do and plot k_means -> test classification models
    hierchical_plot_data: do and plot hierchical clusters - > test classification
models
    Spectral_plot_data: do and plot spectral clusters -> test cllassification
models
'''

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from collections import Counter
from matplotlib.patches import Circle
from matplotlib.lines import Line2D
from scipy.cluster.hierarchy import dendrogram, linkage
from ReduceDimensions import plot_lda
from sklearn.preprocessing import Normalizer
from sklearn.decomposition import PCA
from sklearn.cluster import (SpectralClustering, AgglomerativeClustering,
                             KMeans, FeatureAgglomeration)

from sklearn.neighbors import NearestCentroid
from Classification import evaluate_classifiers



northeast_abbreviations = ['CT', 'ME', 'MA', 'NH', 'RI', 'VT', 'NJ', 'NY', 'PA']
midwest_abbreviations = ['IL', 'IN', 'IA', 'KS', 'MI', 'MN', 'MO', 'NE', 'ND',
'OH', 'SD', 'WI']
southeast_abbreviations = ['DE', 'FL', 'GA', 'MD', 'NC', 'SC', 'VA', 'WV', 'AL',
'KY', 'MS', 'TN']
south_abbreviations = ['AR', 'LA', 'OK', 'TX']
west_abbreviations = ['ID', 'MT', 'WY', 'NV', 'UT', 'CO', 'AZ', 'NM', 'AK', 'WA',
'OR', 'CA', 'HI']
regions_abrv = {
    'Northeast': northeast_abbreviations,
```

```
    'Southeast': southeast_abbreviations,
    'South': south_abbreviations,
    'Midwest': midwest_abbreviations,
    'West': west_abbreviations
}
region_to_color = {
    'Northeast': 'blue',
    'Southeast': 'green',
    'South': 'red',
    'Midwest': 'purple',
    'West': 'orange'
}

state_to_region = {}
for region, states in regions_abrv.items():
    for state in states:
        state_to_region[state] = region









@staticmethod
def k_means_plot_data(X, components=2, reduce = False, n_clusters_options = [25,
15, 6], scalers = [], directory_name = "", title = "L1 Normalization", **kwargs):
    for scale_name, scaler in scalers:
        if reduce:
            norm = Normalizer(norm='l2')
            _X = norm.fit_transform(X)
            pca = PCA(n_components=components)
            _X = pca.fit_transform(_X)

        fig_scatter, axes_scatter = plt.subplots(2, len(n_clusters_options),
figsize=(30, 15))
        fig_centroid, axes_centroid = plt.subplots(1, len(n_clusters_options),
figsize=(21, 6))
        fig_classifer, axes_classifier = plt.subplots(len(n_clusters_options), 3,
figsize=(45, 25))
        plt.subplots_adjust(hspace=0.5, wspace=0.4)
        for idx, n_clusters in enumerate(n_clusters_options):
```

```
            model = KMeans(n_clusters=n_clusters)
            cluster_labels = model.fit_predict(_X)
            if reduce:
                feature_names = [f'component_{i}' for i in range(_X.shape[1])]
                clustered_data = pd.DataFrame(_X, index=X.index,
columns=feature_names)
                clustered_data['cluster'] = cluster_labels
            else:
                clustered_data = pd.DataFrame(_X, index=X.index,
columns=X.columns)
                clustered_data['cluster'] = cluster_labels
            try:
                plot_lda(clustered_data=clustered_data,feature_cols=feature_names
,n_components=components,filename=f'{n_clusters}{scale_name}',directory_name=dire
ctory_name)
            except ValueError:
                print("Cannot do LDA")

            ax_original = axes_scatter[1,idx]
            state_colors = [region_to_color[state_to_region[state]] for state in
X.index]
            scatter = ax_original.scatter(_X[:, 0], _X[:, 1], c=state_colors,
edgecolor='k')
            ax_original.set_title(f'{n_clusters} Clusters | Colored By Region',
fontsize=16)
            ax_original.set_xlabel('PC 1',fontsize=12)
            ax_original.set_ylabel('PC 2',fontsize=12)
            ax_original.set_xticks([])
            ax_original.set_yticks([])
            legend_elements = [Line2D([0], [0], marker='o', color='w',
label=region, markerfacecolor=color, markersize=10) for region, color in
region_to_color.items()]
            ax_original.legend(handles=legend_elements, title="Regions")

            for i, state_label in enumerate(X.index):
                ax_original.annotate(state_label, (_X[i, 0], _X[i, 1]),
textcoords="offset points", xytext=(0,5), ha='center', fontsize=8)

            ax_cluster = axes_scatter[0,idx]
            unique_labels = np.unique(cluster_labels)
            scatter = ax_cluster.scatter(_X[:, 0], _X[:, 1], c=cluster_labels,
edgecolor="k")
            ax_cluster.set_title(f'{n_clusters} Clusters',fontsize=17)
```

```
            legend1 = ax_cluster.legend(*scatter.legend_elements(),
title="Clusters")
            ax_cluster.add_artist(legend1)
            ax_cluster.set_xlabel('PC 1',fontsize=12)
            ax_cluster.set_ylabel('PC 2',fontsize=12)

            for i, state_label in enumerate(X.index):
                ax_cluster.annotate(state_label, (_X[i, 0], _X[i, 1]),
textcoords="offset points", xytext=(0,5), ha='center', fontsize=8)

            centroids = model.cluster_centers_
            centroids_x = centroids[:,0]
            centroids_y = centroids[:,1]
            ax_cluster.scatter(centroids_x, centroids_y,marker = "x",
s=50,linewidths = 1, zorder = 10, c="red")
            ax_original.scatter(centroids_x,centroids_y,marker = "x",
s=50,linewidths = 1, zorder = 10, c="red")

            for i, center in enumerate(centroids):
                points_in_cluster = _X[cluster_labels == i]
                radius = np.max(np.sqrt(np.sum((points_in_cluster - center)**2,
axis=1)))
                circle = Circle(center, radius, color='lightblue', fill=False,
linewidth=3, linestyle='-', alpha=0.8)
                ax_cluster.add_patch(circle)
                circle = Circle(center, radius, color='lightblue', fill=False,
linewidth=3, linestyle='-', alpha=0.8)
                ax_original.add_patch(circle)

            clusters_states = {label: [] for label in unique_labels}
            for state, label in zip(X.index, cluster_labels):
                clusters_states[label].append(state_to_region[state])

            cluster_state_proportions = {}
            for label in unique_labels:
                state_counts = Counter(clusters_states[label])
                total_count = sum(state_counts.values())
                cluster_state_proportions[label] = {state: count / total_count
                                                    for state, count in
state_counts.items()}


            state_colors = {state: color for state, color in
region_to_color.items()}
            ax_cluster_spread = axes_centroid[idx]
```

```
            bottom = np.zeros(len(unique_labels))
            for state in state_colors:
                proportions = [cluster_state_proportions[label].get(state, 0) *
len(clusters_states[label])
                               for label in unique_labels]
                ax_cluster_spread.bar(unique_labels, proportions, bottom=bottom,
color=state_colors[state], label=state)
                bottom += proportions

            legend_elements = [Line2D([0], [0], marker='o', color='w',
label=region, markerfacecolor=color, markersize=10) for region, color in
region_to_color.items()]
            ax_cluster_spread.legend(handles=legend_elements, title="Regions")
            ax_cluster_spread.set_xlabel('Cluster Distribution')
            ax_cluster_spread.set_ylabel('Number of States', labelpad=15)
            ax_cluster_spread.set_title(f'Distribution of States Among
{n_clusters} Clusters')

            _X = scaler.fit_transform(_X)
            evaluate_classifiers(_X, clustered_data, cluster_labels,
feature_names, random_state=42, title=f'KMeans - Clusters {n_clusters} - Scaled
by {scale_name}', directory=directory_name)

        plt.tight_layout()
        fig_scatter.suptitle(f"K-Means Clustering",fontsize=20)
        fig_scatter.savefig(f'{directory_name}/{scale_name}-po-
kmeans{n_clusters_options}.png', dpi=300)
        fig_centroid.savefig(f'{directory_name}/{scale_name}-s-
kmeans_{n_clusters_options}.png', dpi=300)
        fig_classifer.savefig(f'{directory_name}/{scale_name}-m-
kmeans_{n_clusters_options}.png', dpi=300)
        plt.close(fig_scatter)
        plt.close(fig_centroid)
        plt.close(fig_classifer)



@staticmethod
def hierarchical_plot_data(X, components=2, linkage_type = "", n_clusters_options
= [25, 15, 6], reduce = False, scalers = [], do_features = True, directory_name =
""):

    for (scale_name,scaler) in scalers:
        norm = Normalizer(norm='l2')
        _X = norm.fit_transform(X)
```

```
        if reduce:
            pca = PCA(n_components=components)
            _X = pca.fit_transform(_X)
        if do_features:
            feature_agglo = FeatureAgglomeration(n_clusters=components)
            _X = feature_agglo.fit_transform(_X)

        fig_scatter, axes_scatter = plt.subplots(2, len(n_clusters_options),
figsize=(35, 18))
        fig_denospread, axes_deno_spread = plt.subplots(2,
len(n_clusters_options), figsize=(35, 18))
        plt.subplots_adjust(hspace=0.5, wspace=0.4)
        for idx, n_clusters in enumerate(n_clusters_options):
            ax_dendro = axes_deno_spread[0,idx]
            Z = linkage(_X, method=linkage_type)
            dendrogram(Z, ax=ax_dendro, truncate_mode='lastp', p=n_clusters)
            ax_dendro.set_title(f"Dendrogram ({n_clusters} clusters)")
            ax_dendro.set_xlabel('Cluster Index')
            ax_dendro.set_ylabel('Distance')

            model = AgglomerativeClustering(n_clusters=n_clusters,
linkage=linkage_type)
            cluster_labels = model.fit_predict(_X)

            if reduce:
                feature_names = [f'component_{i}' for i in range(_X.shape[1])]
                clustered_data = pd.DataFrame(_X, index=X.index,
columns=feature_names)
                clustered_data['cluster'] = cluster_labels
            else:
                clustered_data = pd.DataFrame(_X, index=X.index,
columns=X.columns)
                clustered_data['cluster'] = cluster_labels
            try:
                plot_lda(clustered_data=clustered_data,feature_cols=feature_names
,n_components=components,filename=f'{n_clusters}{scale_name}',directory_name=dire
ctory_name)
            except ValueError:
                print("Cannot do LDA")
            ax_original = axes_scatter[1, idx]
            state_colors = [region_to_color[state_to_region[state]] for state in
X.index]
            scatter = ax_original.scatter(_X[:, 0], _X[:, 1], c=state_colors,
edgecolor='k')
```

```
            ax_original.set_title(f'{n_clusters} Clusters | Colored By
Region',fontsize=16)
            ax_original.set_xlabel('PC 1',fontsize=12)
            ax_original.set_ylabel('PC 2',fontsize=12)
            legend_elements = [Line2D([0], [0], marker='o', color='w',
label=region, markerfacecolor=color, markersize=10) for region, color in
region_to_color.items()]
            ax_original.legend(handles=legend_elements, title="Regions")
            for i, state_label in enumerate(X.index):
                ax_original.annotate(state_label, (_X[i, 0], _X[i, 1]),
textcoords="offset points", xytext=(0,5), ha='center', fontsize=8)

            ax_scatter = axes_scatter[0, idx]
            unique_labels = np.unique(cluster_labels)
            scatter = ax_scatter.scatter(_X[:, 0], _X[:, 1], c=cluster_labels,
edgecolor='k')
            ax_scatter.set_title(f'{n_clusters} Clusters',fontsize=16)
            ax_scatter.set_xlabel('PC 1',fontsize=12)
            ax_scatter.set_ylabel('PC 2',fontsize=12)
            legend1 = ax_scatter.legend(*scatter.legend_elements(),
title="Clusters")
            ax_scatter.add_artist(legend1)
            for i, state_label in enumerate(X.index):
                ax_scatter.annotate(state_label, (_X[i, 0], _X[i, 1]),
textcoords="offset points", xytext=(0,5), ha='center', fontsize=8)

            clf = NearestCentroid()
            clf.fit(_X, cluster_labels)
            centroids = clf.centroids_
            centroids_x = centroids[:,0]
            centroids_y = centroids[:,1]
            ax_scatter.scatter(centroids_x, centroids_y,marker = "x",
s=50,linewidths = 1, zorder = 10, c="red")
            ax_original.scatter(centroids_x,centroids_y,marker = "x",
s=50,linewidths = 1, zorder = 10, c="red")
            for i, center in enumerate(centroids):
                points_in_cluster = _X[cluster_labels == i]
                radius = np.max(np.sqrt(np.sum((points_in_cluster - center)**2,
axis=1)))
                circle = Circle(center, radius, color='lightblue', fill=False,
linewidth=3, linestyle='-', alpha=0.8)
                ax_scatter.add_patch(circle)
                circle = Circle(center, radius, color='lightblue', fill=False,
linewidth=3, linestyle='-', alpha=0.8)
                ax_original.add_patch(circle)
```

```
            clusters_states = {label: [] for label in unique_labels}
            for state, label in zip(X.index, cluster_labels):
                clusters_states[label].append(state_to_region[state])

            cluster_state_proportions = {}
            for label in unique_labels:
                state_counts = Counter(clusters_states[label])
                total_count = sum(state_counts.values())
                cluster_state_proportions[label] = {state: count / total_count
                                                    for state, count in
state_counts.items()}

            state_colors = {state: color for state, color in
region_to_color.items()}
            ax_cluster_spread = axes_deno_spread[1,idx]
            bottom = np.zeros(len(unique_labels))
            for state in state_colors:
                proportions = [cluster_state_proportions[label].get(state, 0) *
len(clusters_states[label])
                               for label in unique_labels]
                ax_cluster_spread.bar(unique_labels, proportions, bottom=bottom,
color=state_colors[state], label=state)
                bottom += proportions

            legend_elements = [Line2D([0], [0], marker='o', color='w',
label=region, markerfacecolor=color, markersize=10) for region, color in
region_to_color.items()]
            ax_cluster_spread.legend(handles=legend_elements, title="Regions")
            ax_cluster_spread.set_xlabel('Cluster Distribution')
            ax_cluster_spread.set_ylabel('Number of States', labelpad=15)
            ax_cluster_spread.set_title(f'Distribution of States Among
{n_clusters} Clusters')
            _X = scaler.fit_transform(_X)
            evaluate_classifiers(_X, clustered_data, cluster_labels,
feature_names, random_state=42, title=f'Hierarchical - Clusters {n_clusters} -
Scaled by {scale_name}', directory=directory_name)

        plt.tight_layout()
        fig_scatter.suptitle(f"Hierchical Clustering",fontsize=20)
        fig_scatter.savefig(f'{directory_name}/{scale_name}-po-
hierarch_{n_clusters_options}.png', dpi=300)
        fig_denospread.savefig(f'{directory_name}/{scale_name}-ds-
hierarch_{n_clusters_options}.png', dpi=300)
        plt.close(fig_scatter)
```

```
        plt.close(fig_denospread)



@staticmethod
def sepctra_plot_data(X, components=2, n_clusters_options=[2.0,3.0,4.0],
reduce=True, scalers=[], directory_name=""):
    for scale_name, scaler in scalers:
        if reduce:
            norm = Normalizer(norm='l2')
            _X = norm.fit_transform(X)
            pca = PCA(n_components=components)
            _X = pca.fit_transform(_X)
        else:
            _X = X.copy()
        titles = [f"{n} eps {scale_name}" for n in n_clusters_options]

        fig_scatter, axes_scatter = plt.subplots(2, len(n_clusters_options),
figsize=(35, 18))
        fig_denospread, axes_deno_spread = plt.subplots(1,
len(n_clusters_options), figsize=(21, 6))


        for idx, n_clusters in enumerate(n_clusters_options):
            cluster_labels = SpectralClustering(n_clusters=n_clusters,
eigen_solver='arpack').fit_predict(_X)
            if reduce:
                feature_names = [f'component_{i}' for i in range(_X.shape[1])]
                clustered_data = pd.DataFrame(_X, index=X.index,
columns=feature_names)
                clustered_data['cluster'] = cluster_labels
            else:
                clustered_data = pd.DataFrame(_X, index=X.index,
columns=X.columns)
                clustered_data['cluster'] = cluster_labels
            try:
                plot_lda(clustered_data=clustered_data,feature_cols=feature_names
,n_components=components,filename=f'{n_clusters}{scale_name}',
directory_name=directory_name)
            except ValueError:
                print("Cannot do LDA")

            ax_original = axes_scatter[1, idx]
            state_colors = [region_to_color[state_to_region[state]] for state in
X.index]
```

```
            scatter = ax_original.scatter(_X[:, 0], _X[:, 1], c=state_colors,
edgecolor='k')
            ax_original.set_title(f'{n_clusters} Clusters | Colored By
Region',fontsize=16)
            ax_original.set_xlabel('PC 1',fontsize=12)
            ax_original.set_ylabel('PC 2',fontsize=12)
            legend_elements = [Line2D([0], [0], marker='o', color='w',
label=region, markerfacecolor=color, markersize=10) for region, color in
region_to_color.items()]
            ax_original.legend(handles=legend_elements, title="Regions")
            for i, state_label in enumerate(X.index):
                ax_original.annotate(state_label, (_X[i, 0], _X[i, 1]),
textcoords="offset points", xytext=(0,5), ha='center', fontsize=8)

            ax_scatter = axes_scatter[0, idx]
            unique_labels = np.unique(cluster_labels)
            scatter = ax_scatter.scatter(_X[:, 0], _X[:, 1], c=cluster_labels,
edgecolor='k')
            ax_scatter.set_title(f'{n_clusters} Clusters',fontsize=16)
            ax_scatter.set_xlabel('PC 1',fontsize=12)
            ax_scatter.set_ylabel('PC 2',fontsize=12)
            legend1 = ax_scatter.legend(*scatter.legend_elements(),
title="Clusters")
            ax_scatter.add_artist(legend1)
            for i, state_label in enumerate(X.index):
                ax_scatter.annotate(state_label, (_X[i, 0], _X[i, 1]),
textcoords="offset points", xytext=(0,5), ha='center', fontsize=8)

            _X = scaler.fit_transform(_X)
            evaluate_classifiers(_X, clustered_data, cluster_labels,
feature_names, random_state=42, title=f'Spectral - Clusters {n_clusters} - Scaled
by {scale_name}', directory=directory_name)


            clusters_states = {label: [] for label in unique_labels}
            for state, label in zip(X.index, cluster_labels):
                clusters_states[label].append(state_to_region[state])
            cluster_state_proportions = {}
            for label in unique_labels:
                state_counts = Counter(clusters_states[label])
                total_count = sum(state_counts.values())
                cluster_state_proportions[label] = {state: count / total_count
                                                    for state, count in
state_counts.items()}
```

```
            state_colors = {state: color for state, color in
region_to_color.items()}
            ax_cluster_spread = axes_deno_spread[idx]
            bottom = np.zeros(len(unique_labels))
            for state in state_colors:
                proportions = [cluster_state_proportions[label].get(state, 0) *
len(clusters_states[label])
                               for label in unique_labels]
                ax_cluster_spread.bar(unique_labels, proportions, bottom=bottom,
color=state_colors[state], label=state)
                bottom += proportions

            legend_elements = [Line2D([0], [0], marker='o', color='w',
label=region, markerfacecolor=color, markersize=10) for region, color in
region_to_color.items()]
            ax_cluster_spread.legend(handles=legend_elements, title="Regions")
            ax_cluster_spread.set_xlabel('Cluster Distribution')
            ax_cluster_spread.set_ylabel('Number of States', labelpad=15)
            ax_cluster_spread.set_title(f'Distribution of States Among
{n_clusters} Clusters')
            print(f"Clusters {cluster_labels}")

        plt.tight_layout()
        fig_scatter.suptitle(f"Spectral Clustering",fontsize=20)
        fig_scatter.savefig(f'{directory_name}/{scale_name}-po-
spectra_{n_clusters_options}.png', dpi=300)
        fig_denospread.savefig(f'{directory_name}/{scale_name}-ds-
spectral_{n_clusters_options}.png', dpi=300)
        plt.close(fig_scatter)
        plt.close(fig_denospread)


'''
ReduceDimensions - House functions related to dimensionality reduction -
currently only plot_lda

'''

import json
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import matplotlib.pyplot as plt


@staticmethod
```

```
def plot_lda(clustered_data, feature_cols, target_col='cluster', n_components=2,
filename = "", directory_name = ""):
    try :
        lda = LinearDiscriminantAnalysis(n_components=n_components)
        X_lda = lda.fit_transform(clustered_data[feature_cols],
clustered_data[target_col])

        plt.figure(figsize=(12, 6))
        scatter = plt.scatter(X_lda[:, 0], X_lda[:, 1],
c=clustered_data[target_col], edgecolor='k', alpha=0.7)
        plt.title('LDA of Clustered Data',fontsize=14)
        plt.xlabel('LDA Component 1', fontsize=10)
        plt.ylabel('LDA Component 2', fontsize=10)

        plt.colorbar(scatter, label='Cluster Label')
        plt.grid(True)

        plt.savefig(f'{directory_name}/{filename}lda.png')
        plt.close()

    except IndexError:
        print(f"Cannot plot LDA for {clustered_data}")

'''
Class to compare, train, and test classification models with a given set and
scaler stored in the classification model
    note: testing on a new dataset requires a new Classification object

Function definitions:
    def    compare_classifiers: compares classifiers with dataset associated with
a Classification object by dropping a feature in X_predictor
    def    train : train a single classifier given a feature X_predictor, returns
a classification model trained on the specification
    def    predict : make a prediction on the trained classification model, make
a prediction on a trained classification model with a testing set
'''
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.ensemble import (RandomForestClassifier, GradientBoostingClassifier)
from sklearn.linear_model import LogisticRegression
import seaborn as sns
```

```python
@staticmethod
def evaluate_classifiers(X, clustered_data, y, feature_names, train_sizes=[0.2, 0.5, 0.8], random_state=42, axes = None, m_axes = None, title = "", directory = ""):
    classifiers = {
        'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=20, random_state=42),
        'Gradient Boosting': GradientBoostingClassifier(random_state=42, n_estimators=200, ),
        'Logistic Regression': LogisticRegression(max_iter=1000, penalty='l2'),
        "Gaussian Naive Bayes": GaussianNB(var_smoothing=1e-2),
    }

    fig, axs = plt.subplots(2, 3, figsize=(20, 15))

    for i, train_size in enumerate(train_sizes):
        acc_scores = []
        f1_scores = []
        prec_scores = []
        rec_scores = []
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=1-train_size, random_state=random_state)
        for name, clf in classifiers.items():
            clf.fit(X_train, y_train)
            predictions = clf.predict(X_test)
            acc_scores.append(accuracy_score(y_test, predictions))
            f1_scores.append(f1_score(y_test, predictions, average='macro', zero_division=0))
            prec_scores.append(precision_score(y_test, predictions, average='macro', zero_division=0))
            rec_scores.append(recall_score(y_test, predictions, average='macro', zero_division=0))

        axs[0, i].plot(list(classifiers.keys()), acc_scores, label='Accuracy')
        axs[0, i].plot(list(classifiers.keys()), f1_scores, label='F1 Score')
        axs[0, i].set_title(f'Training Size: {train_size}')
        axs[0, i].legend()
        axs[1, i].plot(list(classifiers.keys()), prec_scores, label='Precision')
        axs[1, i].plot(list(classifiers.keys()), rec_scores, label='Recall')
        axs[1, i].set_title(f'Training Size: {train_size}')
        axs[1, i].legend()

    for ax in axs[1]:
        ax.set_xlabel('Classifiers',fontsize=10)
```

```
    for ax in axs[:, 0]:
        ax.set_ylabel('Scores', fontsize=10)

    fig.suptitle(f"Classifiction | {title}", fontsize=20)

    plt.subplots_adjust(left=0.1,
                        bottom=0.1,
                        right=0.9,
                        top=0.9,
                        wspace=0.4,
                        hspace=0.4)
    plt.tight_layout()
    print("GoGoGoGoGO")
    fig.savefig(f"{directory}/{title}.png")
    plt.close(fig)
```